

Permissions

CRM/ERP Platform Documentation

Permissions

The permission system has three layers that compose in a specific order.

Resolution Order

1. SUPER_ADMIN role → bypass ALL checks
2. Role-level permission (Permission table)
3. Department-level override (Department.permissions JSON)
4. User-level override (UserPermissionOverride table)

A more specific override always wins over a broader one. User overrides can grant OR revoke access.

Layer 1: Role Permissions

Stored in the `permissions` table. One row per `(organizationId, role, moduleId)`.

```
{
  canCreate: boolean
  canRead: boolean
  canUpdate: boolean
  canDelete: boolean
  canExport: boolean
  canPrint: boolean
  canApprove: boolean
  canManage: boolean
}
```

```
canFormBuilder: boolean
canDashboard: boolean
canAnalytics: boolean
canSettings: boolean
}
```

Managed via `POST /permissions/bulk` or the UI at **Settings** → **Permissions**.

Default seed: When a new module is created, `POST /permissions/seed/:moduleId` creates default rows for all roles.

Layer 2: Department Override

`Department.permissions` is a JSON field:

```
{
  "canCreate": true,
  "canDelete": false,
  "moduleId": "..."
}
```

Set via `PATCH /departments/:id/permissions`.

Layer 3: User Override

`UserPermissionOverride` table — per user, per module:

```
{
  userId: string
  moduleId: string
  canView?: boolean    // null = inherit from role
  canCreate?: boolean
  canEdit?: boolean
  canDelete?: boolean
  canExport?: boolean
}
```

```

canImport?: boolean
canDashboard?: boolean
canAnalytics?: boolean
canWorkflow?: boolean
canForms?: boolean
canStudio?: boolean
isActive: boolean
expiresAt?: Date      // optional time-limited grant
reason?: string
grantedById: string
}

```

Only `SUPER_ADMIN` can set user overrides via `POST /users/:id/permission-overrides` .

Current User Permissions

`GET /users/me/permissions` returns the resolved permission set for the logged-in user across all modules, after applying all three layers.

Use this endpoint on the frontend to show/hide UI elements:

```
const { canCreate, canDelete } = permissions[moduleId] ?? {};
```

Form Permissions

Forms have their own permission model (`FormPermission` table):

```

{
  formId: string
  role: string
  canView: boolean
  canSubmit: boolean
  canEdit: boolean
  canDelete: boolean
  canShare: boolean
}

```

```
canManageBuilder: boolean
}
```

Additionally, `Form.sharedUsers`, `Form.sharedDepts`, and `Form.sharedRoles` are JSON arrays for fine-grained sharing.

Dashboard Permissions

Dashboards use inline JSON columns (no separate table):

- `sharedRoles: Json` — array of role strings
- `sharedDepartments: Json` — array of department IDs
- `sharedUsers: Json` — array of user IDs

A dashboard is visible to a user if their role, department, or user ID appears in those arrays, or if `isPublic: true`.

Portal Permissions

Portal users have separate roles (`portalRole: "user" | "admin"`) and are gated by `isPortalAdmin` flag for builder access. Portal users cannot access any CRM route.

SUPER_ADMIN

The `SUPER_ADMIN` role bypasses all permission checks. It can:

- Access all organizations
- Create/delete any resource
- Grant/revoke user overrides
- Suspend/lock users and organizations

Only the first registered user gets this role automatically. Additional SUPER_ADMINs must be set by an existing SUPER_ADMIN via `PATCH /users/:id`.